

PROBLEM SET 4: PENJADWALAN CPU & MLFQ

BERBASIS TAKSONOMI BLOOM (LEVEL C1 S.D. C6)

Mata Kuliah: Sistem Operasi (INF-201) – Program Studi Informatika UNIB

SOAL LEVEL C1 – C2: KONSEPTUAL DASAR

1. Apakah yang dimaksud dengan fenomena *Convoy Effect* pada algoritma Penjadwalan FCFS? Jelaskan dampaknya terhadap utilisasi perangkat I/O dan rata-rata waktu tunggu!
2. Terbukti secara matematis bahwa algoritma SJF meminimalkan rata-rata waktu tunggu ($\bar{W}$). Mengapa SJF sulit diimplementasikan secara murni pada Sistem Operasi *general-purpose*?

SOAL LEVEL C3 – C4: KALKULASI GANTT CHART & ANALISIS PREEMPTION

3. Diberikan 4 proses yang tiba pada waktu berbeda di *Ready Queue*:

Proses	Waktu Kedatangan (T_{datang})	Burst Time (T_{burst})
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

- a. Gambarkan **Gantt Chart** dan hitung rata-rata waktu tunggu ($\bar{W}$) menggunakan algoritma **SRTF (Shortest Remaining Time First / Preemptive SJF)**!
 - b. Gambarkan **Gantt Chart** dan hitung rata-rata waktu tunggu ($\bar{W}$) menggunakan algoritma **Round Robin (RR)** dengan *Time Quantum* $q = 2$!
 - c. Analisis mengapa proses P_1 mengalami waktu tunggu yang sangat berbeda antara SRTF dan Round Robin!
4. Jika nilai *Time Quantum* (q) pada Round Robin diatur sangat kecil (misal $q = 1 \mu\text{s}$ sementara overhead context switch adalah $10 \mu\text{s}$), hitung persentase kapasitas CPU yang terbuang sia-sia (*CPU waste percentage*) dan jelaskan implikasinya terhadap performa sistem!

SOAL LEVEL C5: EVALUASI ARSITEKTUR MLFQ

5. Pada algoritma **Multilevel Feedback Queue (MLFQ)**, jelaskan bagaimana program jahat (*malicious process*) dapat memanipulasi aturan *time-slice* klasik (*gaming the scheduler*) agar tetap mendominasi antrian prioritas tertinggi! Bagaimana Aturan ke-4 modern (*Accounting with Rule 4*) mampu menutup celah eksploitasi tersebut?

SOAL LEVEL C6: DESAIN ATURAN PRIORITY AGING ALGORITHM

6. Rancanglah sebuah algoritma *Priority Aging* secara matematis/pseudocode yang menaikkan tingkat prioritas proses secara dinamis terhadap waktu tunggu W , sedemikian rupa

sehingga proses dijamin tidak akan mengalami *Starvation* lebih dari $T_{\max}$ detik terlepas dari berapa banyak proses berprioritas tinggi yang terus berdatangan ke sistem!